LockedIn — Complete A-to-Z Viva Study Guide

1. 60-Second Project Pitch (Start Your Viva With This)

> **"LockedIn is a full-stack peer-to-peer academic skill-exchange and collaborative study platform built with React (Vite) on the frontend and Java Spring Boot on the backend. Instead of paying money for tutoring, students exchange knowledge using a time-banking credit system: every new student gets 10 welcome credits, earns credits by teaching peers or solving doubts, and spends credits when learning. It features an automated skill-matching engine (prioritizing mutual matches), shared study groups with unique invite IDs, syllabus progress tracking, a collaborative doubt-solving forum and wiki, and built-in real-time WebRTC peer-to-peer video and audio sessions."**

2. Complete Tech Stack & Why Each Tool Was Used

Layer	Technology	Why We Used It (Viva Answer)
Frontend Framework	**React 18 + Vite**	Component-based single-page application (SPA) with fast Hot Module Replacement (HMR) and optimized production bundling.
Frontend Routing	**React Router v6**	Handles client-side navigation (`/login` , `/dashboard` , `/skills` , `/matches` , `/groups/:groupId` , `/credits`) without full page reloads.
HTTP Client	**Axios**	Centralizes API calls to the Spring Boot backend and uses an **interceptor** to automatically attach the `Bearer <JWT>` token to every request.
Real-Time Video/Audio	**WebRTC (`RTCPeerConnection` + `getUserMedia`)**	Streams camera video and microphone audio directly peer-to-peer between browsers using Google STUN servers (`stun.l.google.com:19302`) and a Spring Boot signaling relay.
Backend Framework	**Java 17 + Spring Boot 3**	Provides layered enterprise architecture (Controller → → Service → → Repository), dependency injection, and embedded Tomcat server.
Security & Auth	**Spring Security + JWT (`jjwt` + `BCrypt`)**	Stateless authentication. Passwords are hashed using `BCryptPasswordEncoder` ; logged-in users receive a signed HMAC-SHA256 JWT token valid for 24 hours.
Database & ORM	**Spring Data JPA (Hibernate) + H2 File DB / PostgreSQL**	Maps Java `@Entity` classes directly to relational SQL tables (`ddl-auto=update`) with zero boilerplate SQL.
Deployment	**Docker + Render (Backend) & Vercel (Frontend)**	Backend is containerized via a multi-stage `Dockerfile` on Render; Frontend is deployed on Vercel's global CDN with SPA rewrite rules (`vercel.json`).

3. System Architecture & End-to-End Flow

How a Request Flows from Click to Database:

1. **User Action:** The user clicks a button in React (e.g., **"Add skill"** on).

2. **Axios Request Interceptor:** reads `lockedin_token` from `localStorage` and attaches `Authorization: Bearer <token>` to the HTTP request header.
3. **Spring Security Filter:** intercepts the request before it reaches any controller, verifies the JWT signature via , extracts the `userId`, and stores it in `SecurityContextHolder`.
4. **Controller Layer:** receives the JSON body (`@RequestBody UserSkillRequest`) and the logged-in user's ID (`@AuthenticationPrincipal Long userId`).
5. **Service Layer:** executes the business logic (finds or creates the `Skill`, checks if the user already has that skill, and links it to the `User`).
6. **Repository & Database Layer:** executes the SQL `INSERT` / `UPDATE` via Hibernate and returns the saved entity as JSON back to React, which updates state (`setSkills`) and re-renders the UI immediately.

4. Backend Folder & File-by-File Breakdown (`lockedin-backend`)

A. Root Configuration Files

- : Maven configuration defining dependencies (`spring-boot-starter-web`, `spring-boot-starter-data-jpa`, `spring-boot-starter-security`, `spring-boot-starter-validation`, `jjwt`, `lombok`, `h2`, `postgresql`).
- : Two-stage Docker build: Stage 1 uses Maven + JDK 17 to compile `backend-0.1.0.jar`; Stage 2 runs the JAR on a lightweight JRE 17 container on port `8080`.
- : Configures server port `8080`, persistent H2 database (`jdbc:h2:file:./data/lockedindb`), Hibernate `ddl-auto=update`, and JWT secret/expiration (`86400000` ms = 24 hours).
- : Main entry point (`@SpringBootApplication`) and `@EnableScheduling` for scheduled background jobs.

B. `config/` & `security/` (Authentication, CORS & WebSockets)

- :
    - Configures **CORS** so the Vercel frontend can call the Render backend.
    - Disables CSRF (since we use stateless JWTs) and sets session policy to `STATELESS`.
    - Allows public access to `/api/auth/**` and `/api/signal/**`, while requiring authentication for all other `/api/**` routes.
    - Provides the `BCryptPasswordEncoder` bean.
- : Generates and validates JWT tokens using HMAC-SHA256 (`Keys.hmacShaKeyFor`). Embeds `userId` as the token `subject` and `email` + `name` as custom claims.

- : Extends `OncePerRequestFilter`. Reads the `Authorization: Bearer ...` header on every request, validates the token, ensures the user exists in the database, and populates Spring Security's `UsernamePasswordAuthenticationToken`.
- : Configures STOMP WebSocket message broker endpoints (`/ws`, `/topic`, `/app`).

C. `entity/` (Database Tables — 22 Classes)

- : Stores `id`, `email`, `name`, `passwordHash`, `department`, `yearOfStudy`, `bio`, `credits` (default `10`), and `createdAt`.
- & : `Skill` stores unique skill names (e.g., `Java`, `DBMS`); `UserSkill` is the join table linking a `User` to a `Skill` with (`TEACH` or `LEARN`) and (`BEGINNER`, `INTERMEDIATE`, `ADVANCED`).
- : Stores accepted tutor-learner pairs (`userA` = teacher, `userB` = learner, `skill`, = `MUTUAL` or `ONE_DIRECTIONAL`, = `PENDING` / `ACCEPTED`).
- & : `StudyGroup` stores `id` (the shareable Group ID), `name`, `subject`, `description`, `maxMembers`, and `createdBy`. `GroupMember` tracks which users joined which group and their (`LEADER` or `MEMBER`).
- & : Stores topics in a study group's syllabus and tracks each member's (`NOT_STARTED`, `IN_PROGRESS`, `COMPLETED`).
- & : Stores questions posted inside a group (`title`, `body`, `resolved`) and peer answers/clarifications (`body`, `accepted`).
- : Stores shared study notes (`title`, `content`) pinned to a group's Wiki.
- & : Represents booked video study sessions (`ONE_ON_ONE` or `GROUP`, `topic`, `scheduledAt`, `durationMinutes`, `meetingLink`,) and collaborative session notes.
- : Ledger recording every credit change (`INITIAL_GRANT`, `EARNED_TEACHING`, `SPENT_LEARNING`) with `amount`, `description`, and `createdAt`.
- , , & : Store 1–5 star tutor ratings, weekly availability windows, and study reminders.

D. `dto/` (Data Transfer Objects — 9 Classes)

DTOs decouple the API contract from internal database entities and validate incoming inputs (`@NotBlank`, `@Email`):

- , ,
- ,
- ,
- , ,

E. `service/` & `controller/` (Business Logic & REST API Endpoints)

1. **Authentication** — &
   - `POST /api/auth/register`: Hashes password with BCrypt, saves user with 10 initial credits, records an `INITIAL_GRANT` transaction, and returns JWT.
   - `POST /api/auth/login`: Verifies email & BCrypt password hash and returns JWT.
2. **Skills Management** — &
   - `POST /api/skills/me`: Adds or updates a skill (`TEACH` or `LEARN`) for the logged-in user.
   - `GET /api/skills/me`: Lists the logged-in user's skills.
   - `DELETE /api/skills/me/{userSkillId}`: Removes a skill from the user's list.
3. **Peer Matching Engine** — &
   - `GET /api/matches/suggestions`: Finds tutors who `TEACH` skills the user wants to `LEARN` (and vice versa), checks if the match is `MUTUAL` (User A teaches what User B wants, and User B teaches what User A wants) or `ONE_DIRECTIONAL`, and sorts `LEARN` + `MUTUAL` matches to the top.
   - `GET /api/matches/tutors`: Lists all students currently teaching skills on the platform.
   - `POST /api/matches/accept` & `GET /api/matches/accepted`: Saves and retrieves accepted peer matches.
4. **Study Groups** — &
   - `POST /api/groups`: Creates a group, assigns a unique Group ID, and adds creator as `LEADER`.
   - `POST /api/groups/{groupId}/join`: Lets any student join using the shared Group ID (enforcing `maxMembers`).
   - `GET /api/groups/my`: Returns all groups the logged-in user belongs to.
5. **Group Doubts & Clarifications** — &
   - `POST /api/groups/{groupId}/doubts` & `GET /api/groups/{groupId}/doubts`: Posts and lists doubts with all their answers.
   - `POST /api/groups/{groupId}/doubts/{doubtId}/answers`: Posts an answer/clarification to a doubt.
   - `POST /api/groups/{groupId}/doubts/{doubtId}/resolve`: Marks a doubt as resolved.
6. **Group Syllabus & Wiki** — , , ,
   - Manages group syllabus topics, per-member completion status (`NOT_STARTED`, `IN_PROGRESS`, `COMPLETED`), and shared Wiki articles.

7. **Sessions & Credits** — , , ,
    - Books study sessions (`POST /api/sessions`), completes sessions (`POST /api/sessions/{id}/complete` which transfers credits to the tutor), and returns credit balance + transaction history (`GET /api/credits/me`).
8. **WebRTC Video Signaling** —
    - `POST /api/signal/{room}` & `GET /api/signal/{room}?since={seq}` : In-memory signaling relay that exchanges WebRTC SDP `offer` , `answer` , and `ice` candidates between participants in the same video room.

5. Frontend Folder & File-by-File Breakdown (`lockedin-frontend`)

- : Mounts React into `#root` wrapped in `<BrowserRouter>` .
- : Defines all routes (`/login` , `/register` , `/dashboard` , `/skills` , `/matches` , `/groups` , `/groups/:groupId` , `/credits`).
- : Configures Axios `baseURL` (`https://lockedin-enb3.onrender.com`), attaches `Authorization: Bearer <token>` , and redirects to `/login` on `401/403` .
- : Top navigation bar displaying page links, the logged-in user's name, and Logout button.
- : Route guard that checks for `lockedin_token` in `localStorage` and redirects unauthenticated users to `/login` .
- : Full WebRTC video & audio call component:
    - Calls `navigator.mediaDevices.getUserMedia({ video: true, audio: true })` to start camera and mic.
    - Creates `RTCPeerConnection` per peer, exchanges SDP offers/answers and ICE candidates via `/api/signal/{room}` , and renders live `<video>` tiles with Mute/Unmute and Camera On/Off controls.
- & : Authentication forms that store `lockedin_token` , `lockedin_userId` , and `lockedin_name` in `localStorage` .
- : Overview hub showing the user's credit balance, active study groups, skills count, and top suggested matches.
- : Lets users add/remove `TEACH` and `LEARN` skills with proficiency levels and previews matching tutors.
- : Displays **Matches to Learn**, **Accepted Matches**, **Students Who Want to Learn From You**, **All Available Tutors**, and launches 1-on-1 WebRTC video calls.

- : Creates study groups, generates a copyable **Group ID badge** (`Group ID: #1`), and lets other students join by pasting that Group ID.
- : Inside a study group:
    1. **Live Video Sessions:** Book a session and click **Join Live Video Call** to connect on camera & audio.
    2. **Doubts & Clarifications:** Post questions, write clarifications/answers, and mark doubts as resolved.
    3. **Syllabus Tracker:** Add topics and toggle status (`Not started` → → `In progress` → → `Completed`).
    4. **Group Wiki:** Create shared study notes.
- : Displays current credit balance and full transaction ledger.
- : Custom CSS using a sleek **Dark Theme** (`#0B0F19` background, `#111827` cards, `#6366F1` indigo primary accents, `#38BDF8` cyan highlights, `#34D399` emerald badges).

6. Top 10 Viva Questions & Exact Answers to Give

1. **Q: How does authentication work in your project?**
    - **A:** We use **stateless JWT (JSON Web Token) authentication** with Spring Security. On registration, the user's password is encrypted using **BCrypt** (`BCryptPasswordEncoder`) before saving to the database. On login, the server verifies the password and returns a signed HMAC-SHA256 JWT token. React stores this token in `localStorage`, and an Axios interceptor attaches `Authorization: Bearer <token>` to every API call, which `JwtAuthFilter` validates on the backend.
2. **Q: How does the skill-matching algorithm work?**
    - **A:** In `MatchingService.java`, we fetch the logged-in user's `LEARN` skills (`SkillType.LEARN`) and compare them against all other users' `TEACH` skills (`SkillType.TEACH`) using case-insensitive and substring matching. For each candidate tutor, we also check if the current user can teach any skill that the candidate wants to learn. If yes, it is tagged as a **`MUTUAL`** match; otherwise, **`ONE_DIRECTIONAL`**. Mutual matches are sorted to the top.
3. **Q: How do live video and audio calls work between two users?**
    - **A:** We use **WebRTC (`RTCPeerConnection`)**. First, `navigator.mediaDevices.getUserMedia({ video: true, audio: true })` captures the local camera and microphone stream. To connect two browsers across the internet, they need to exchange network details (ICE candidates) and media descriptions (SDP Offer and Answer). Our `SignalController`

(`/api/signal/{room}`) acts as the signaling server where peers announce `join`, send an SDP `offer`, reply with an SDP `answer`, and exchange `ice` candidates via Google's STUN server (`stun:stun.l.google.com:19302`). Once negotiated, video and audio flow directly peer-to-peer.

4. **Q: Why did you need `vercel.json` in the frontend?**
    - **A:** Because React Router handles routing in the browser (Single Page Application). Without `vercel.json`, refreshing a URL like `/login` or `/groups/1` on Vercel would look for a physical file named `login` on the server and return `404`. The rewrite rule `"source": "/(.*)", "destination": "/index.html"` tells Vercel to serve `index.html` for all routes so React Router can render the right page.
5. **Q: How does the Group ID sharing work?**
    - **A:** When a user creates a group (`POST /api/groups`), the database generates a unique primary key (`group.id`) and assigns the creator as `GroupRole.LEADER`. The UI displays this ID with a **Copy ID** button. Any other student can enter that ID on the Groups page, which calls `POST /api/groups/{groupId}/join`, checks that the group hasn't exceeded `maxMembers` and that the user isn't already a member, and inserts a `GroupMember` row.
6. **Q: How does the Credit / Time-Banking system work?**
    - **A:** Every new user starts with **10 credits** (`INITIAL_GRANT`). When a tutor conducts and completes a study session (`SessionService.completeSession`), credits are awarded (`EARNED_TEACHING`) proportional to the session duration, and learners spend credits (`SPENT_LEARNING`). Every change is recorded in the `credit_transactions` ledger table so there is an auditable history.
7. **Q: What is the difference between `@Entity`, `DTO`, and `Repository` in your backend?**
    - **A:**
        - `@Entity` represents a database table mapped by Hibernate/JPA (e.g., `User`, `StudyGroup`).
        - `Repository` is an interface extending `JpaRepository` that performs CRUD database queries without writing manual SQL.
        - `DTO` **(Data Transfer Object)** defines the exact JSON structure sent in requests/responses (e.g., `LoginRequest`, `MatchSuggestionResponse`) so we never expose sensitive fields like `passwordHash`.
8. **Q: Why did you disable CSRF in `SecurityConfig.java`?**
    - **A:** CSRF attacks exploit browser cookies that are automatically sent with requests. Since our API is **stateless** (`SessionCreationPolicy.STATELESS`) and uses a `Bearer` JWT token

stored in `localStorage` sent via explicit `Authorization` headers (not cookies), CSRF protection is not needed for stateless REST APIs.

9. **Q: How did you deploy the backend and frontend separately?**
    - **A:** The backend is containerized using a multi-stage **Dockerfile** (Maven + JDK 17 to build the JAR, then Eclipse Temurin JRE 17 to run it) and deployed on **Render**. The frontend is built with Vite and deployed on **Vercel**, connected to the Render backend via `axios.js` and CORS configuration in `SecurityConfig.java`.
10. **Q: What happens if a doubt is posted inside a Study Group?**
    - **A:** The frontend sends `POST /api/groups/{groupId}/doubts` with `title` and `body`. `DoubtService` links the `Doubt` to the `StudyGroup` and the author (`User`). All group members immediately see the doubt on the Group Detail page, can submit clarifications (`POST /api/groups/{groupId}/doubts/{doubtId}/answers`), and mark the doubt as **Resolved** (`POST /api/groups/{groupId}/doubts/{doubtId}/resolve`).